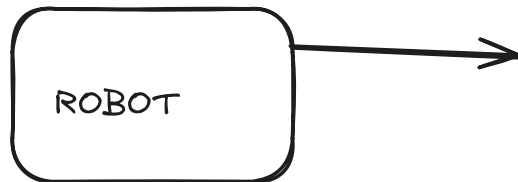


Topic: Javascript Control Structures
Mentor: Sushant Goyal
Date: 02-Aug-25

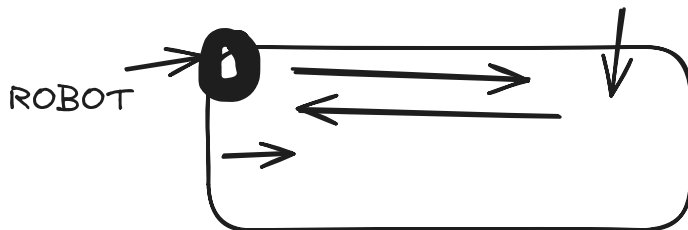
Topics for Today:

1. What are control structures
2. Functions
3. If --- else if ---- else
4. While --- do-while
5. For loops
- 6.



Instructions

1. It can clean floors
2. it can communicate



1. Decide between two or more paths
2. Repeat certain steps multiple times
3. Stop doing something when a condition is met.

Why we require control structures:

1. Code smarter (it can react)
2. shorter (reusable)
3. Flexible (can perform different things depending on situation)

Decision Making:

1. This is like a condition which is based on the goal or current situation
2. Code chooses a path based on the destination or the result

Loops:

1. This is like repeating a set of steps until a certain condition is met.

Branching or Jumps:

1. Skipping some steps to reach a statement based on the options

1. Decision Making

a. if statements - this is the main statement controlling the decision

(if I study JS i can be a web developer)

b. else if statements - Additional conditions - this is optional as well (if I learn about blockchain i can be a block chain or web3 developer)

c. else statements - if all the above conditions doesn't meet - this is optional (I am a developer)

```
if(roomTemp == 35) 'turn on the AC'
```

```
elseif(roomTemp <15) 'turn on the heater'
```

```
else 'turn off the ac'
```

1. == - check the left and right values

2. === - check the left and right values along with their data types

3. > - check if the left expression is greater than the right expression

4. < check if the left expression is smaller than the right expression

5. >= - check if the left expression is greater than or equal to the right expression

- 5. $\geq$ - whether the left expression is greater than or equal to the right expression
- 6. $\leq$ - whether the left expression is less than or equal to the right expression
- 7. $\neq$ - left expression is not equal to right expression

2. Loops

Loops allows us to automate repetitive steps by running a block multiple times

- a. until a condition is met
- b. for a fixed number of iterations.

- 1. while loop:
as long as the condition is true it will remain in execution phase
- 2. do - while loop
it will run once and then check if the condition is met
- 3. for loop
it runs for a specific number of times

// Example for while loop

```
let i = 1
while(i<=5) {
  console.log(i)
  i++ // incrementing the value for next iteration
}
```

Iterations

- 1. $i=1$, print -1
- 2. $i=2$, print-2
- 3. $i=3$, print - 3
- 4. $i=4$, print - 4
- 5. $i=5$, print - 5
- 6. $i = 6$, but it will not be executed because i is not less than or equal to 5

// Used when the number of iterations are not known

// Example for do while loop

// we perform the steps then check for the condition

let x = 1

```
do {  
  console.log(x)  
  x++  
}  
while(x<=5)
```

iterations:

1. x = 1, print then increment (2<=5)-> then check if is valid
2. x = 2, print then increment (3<=5)-> then check if is valid
3. x = 3, print then increment (4<=5)-> then check if is valid
4. x = 4, print then increment (5<=5)-> then check if is valid
5. x = 5, print then increment (6<=5)-> then check if is valid

// Example for FOR loop

```
for(let i=0;i<5;i++) {  
  console.log(i)  
}
```

// set the iterator, the condition, then iteration update

// Used when we have set number of iterations or number of iterations are known

3. Branching or Jumps

Switch statement

```
let theme = 'christmas'
```

```
switch (theme) {  
  case 'christmas':  
    console.log('Play christmas carol')  
    break  
  
  case 'new year':  
    console.log('Play new year music')  
    break  
  
  case 'Holi':  
    console.log('Play holi music')  
    break  
  
  default:  
    console.log('Party mood')  
    break  
}
```

```
let grade = 85
```

```
switch (true) {  
  case (grade > 90):  
    console.log('Grade A')  
    break  
  
  case grade > 80:  
    console.log('Grade B')  
    break  
}
```

```
case grade > 70:
  console.log('Grade C')
  break

case 85:
  console.log('Match found')
  break

default:
  console.log('Grade D')
  break
}
```

4. Functions:

It is a named code block that performs specific set of instructions which can be reused.

- a. it can take an input
- b. it can return a value
- c. it can be called anywhere

```
function greet() {
  console.log('Hello World')
}
```

```
greet()
```

```
var a = 'some value'
```

```
console.log(a)
```

Placing an order on E-commerce

1. Select the products

2. GO to Cart
 3. Make Payment
 4. Trigger the payment gateway with the payment information
 - i. amount
 - ii. account that will be receiving the payment
- // Step 4 is a function which takes 2 input mention (i) and (ii)
5. A payment gateway opens
 - i. it waits for an approval from the initiator or user
 - ii. it will check if the payment or the amount is deducted and credited.
 - iii. it will return a flag whether the payment is successful or not.
 6. The control comes again to the E-commerce.
 7. E-commerce checks for the flag received. (We are at step 4, we came back to step 4)
 8. If successful the order is placed else it is rejected.

Feature Why it matters

1. Reusability Write once, use anywhere
2. Modularity Break down complex parts into manageable smaller chunks
3. Abstraction Hide details that we don't care about
4. Flow Control Jump to a function, run its code and return the control
5. Recursion If we want to call a function within itself.

Make 2 operations - addition and subtraction
we can use functions, conditional statements or jump statements

defer -> It will wait till the end of HTML rendering, then it will process the script.
async -> it will delegate the execution not necessarily till the end of HTML rendering
integrity -> it checks for the hash key within the script tag and the file / response received

from the server. if it is not the same the browser will refuse to process the script
cross-origin -> it checks if we can access the server either anonymously or by adding
some credentials in the form of cookies